Static Analysis

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

Static

- `grep -r "sscanf[^)]*,[^)]*%s"`

```
static char cs;

...

int ret = sscanf(buf, "%s", &cs);

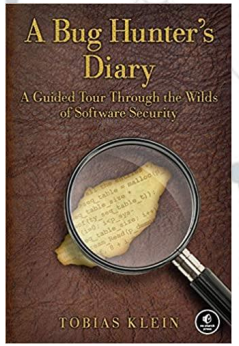
if (ret != 1) {

   accdet_error("..");

   return -1;

}

...
```



Most successful technique

CVE-2016-8472: In MediaTek Kernel Driver

Static

- `grep -r "sscanf[^)]*,[^)]*%s" -> CVE-2016-8472`
  - Along with **2,300** other matches which **are not vulnerabilities (False positives)**.

```
...
char *ptr = “CMD 12”;
char buf[64]
...
sscanf(ptr, "%s", buf);
```

Maximum size could be 6 (less than 64 -> size of buf)

It becomes worse on complex codebases!

	CppCheck	flawfinder	RATS
Qualcomm	18	4,365	693
Samsung	22	8,173	2,244
Huawei	34	18,132	2,301
MediaTek	168	14,230	3,730
Total	**242**	**44,990**	**8,968**

Static

- How does a human find vulnerabilities?

```
void overflow() {
  char *out;
  int in = get_int();
  if (in <= 0) { return; }
  out = malloc(in*sizeof(char*));
  for (i = 0; i < in; i++)
    out[i] = get_string();
}
```

1073741824

0

Static

- How does a human find vulnerabilities?
  - Understands the program and tries to find if any vulnerable conditions are possible.
  - We need a way to analyze the given program or software:
    - Program Analysis -> **Static Program Analysis** or **Static Analysis**

But, computing program properties is undecidable!

```
void foo() {
  int a[2];
  M(X);
  a[3] = 0;
}
```

- Halting Problem: Impossible to say whether a program terminates.
- Proof by contradiction:
  - Yes -> Execution reaches a[3] i.e., program M(X) terminates.
  - No -> Execution does not reach a[3] i.e., program M(X) does not terminate.
- Contradiction: We can say if a program terminates.

Static analysis design choices for vulnerability detection

	Recall=1		Precision=1	
True Result	**Sound**	**Complete**	**Neither sound nor complete**	**Sound and Complete**
Bug	Bug	May or May not be a bug.	May or May not be a bug.	Bug
Not a bug	May or May not be a bug.	Not a bug.	May or May not be a bug.	Not a bug
	false positives No false negatives	No false positives false negatives	false positives false negatives	No false positives No false negatives

Sound Static Analysis

- **Used to be the** popular choice. Why?
  - Guarantees that all bugs will be found.
    - Over Approximation.
    - Caveat: False positives.
  - If a sound static analysis says, there are no bugs*, then we can be sure that the program does not have bugs.

* of specific type.

Designing a Sound Static Analysis

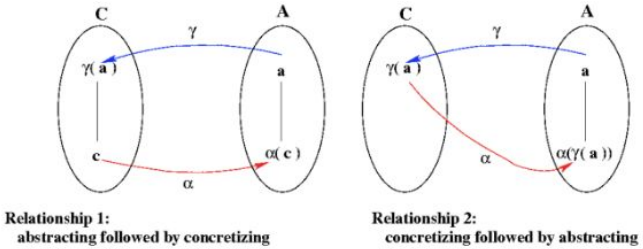
- Guaranteed Termination: Should finish in reasonable time.
- **Over Approximate** program behavior.

Abstract Interpretation

- Interpret the program over abstract states.
- Abstract semantics:
  - How to interpret operations over abstract values.
- Guaranteed Termination (Kleene fixed-point theorem):
  - Galois Connection.
  - Monotonic Transfer functions:
    - The state computed at a program point should never decrease.

Abstract Interpretation

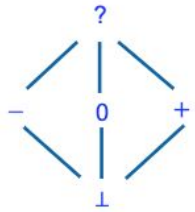
- Galois Connection:
  - Abstraction function ($_{\alpha}$) -> Maps a set of concrete values to abstract value.
  - Concretization function ($_{\gamma}$) -> Maps an abstract value to set of concrete values.
    1. **$\alpha(c) \leq a \Longleftrightarrow c \leq \gamma(a)$**
    2. **$\alpha(\gamma(a)) \leq a$**



Sign Abstract Domain

To handle properties related to integers.

Abstract Values: {-, 0, +, ⊥, ?}



$$\alpha(S) = \begin{cases} 0 & \text{if all elements of S are 0} \\ + & \text{if all elements of S are positive} \\ - & \text{if all elements of S are negative} \\ ? & \text{otherwise} \end{cases}$$

$$\gamma(S) = \begin{cases} \{0\} & \text{if } S = 0 \\ \{\text{pos int}\} & \text{if } S = + \\ \{\text{neg int}\} & \text{if } S = - \\ \{0 \text{ pos neg}\} & \text{if } S = ? \end{cases}$$

ADD	-	0	+	?
-	-	-	?	?
0	-	0	+	?
+	?	+	+	?
?	?	?	?	?

MULT	-	0	+	?
-	+	0	-	?
0	0	0	0	0
+	-	0	+	?
?	?	0	?	?

Divide by Zero Detector

- We do not care about absolute values of integers.
- We **just need to know if a number can be 0 or not**.
- Sign abstract domain provides a decent choice.
- Possible values for numbers: {-, 0, +, ⊥, ?}

```
void main() {
   ...
   if (x > 0) {
      ...1/x.. // x:+
   }
   ...2/x.. // x: ?
}
```

numReq

uests: ?

```
int averageResponseTime(int totalTime, int numRequests) {
  return totalTime / numRequests;
}
```



CVE-2019-14498
A divide-by-zero error exists in VLC media player that can be exploited by a crafted audio file

Data flow analysis

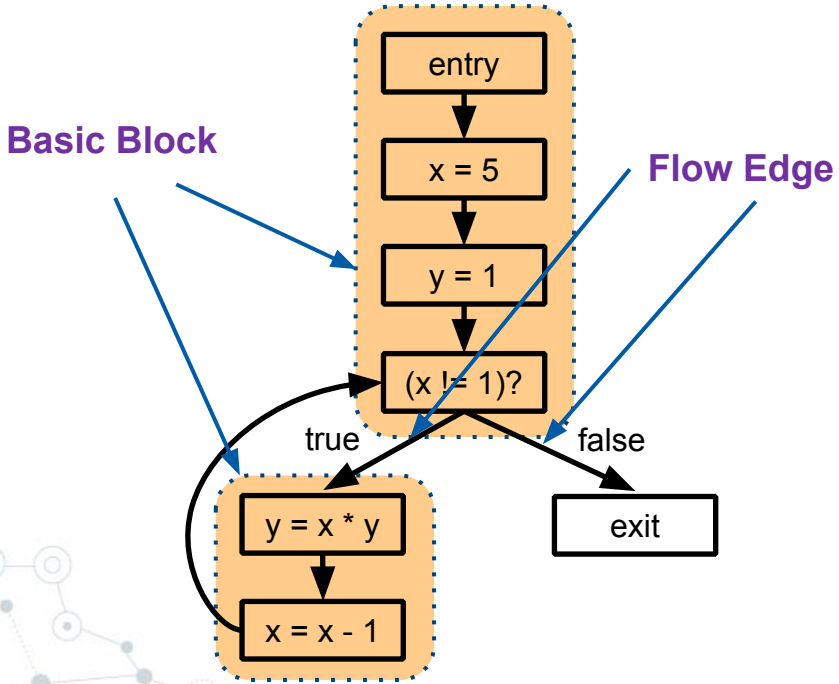
- Most vulnerabilities need reasoning of the flow of data through the program.
  - E.g., user input used as an index into an array => User data flows into index of an array.

- Reasoning about flow of data in programs.

Data flow concepts

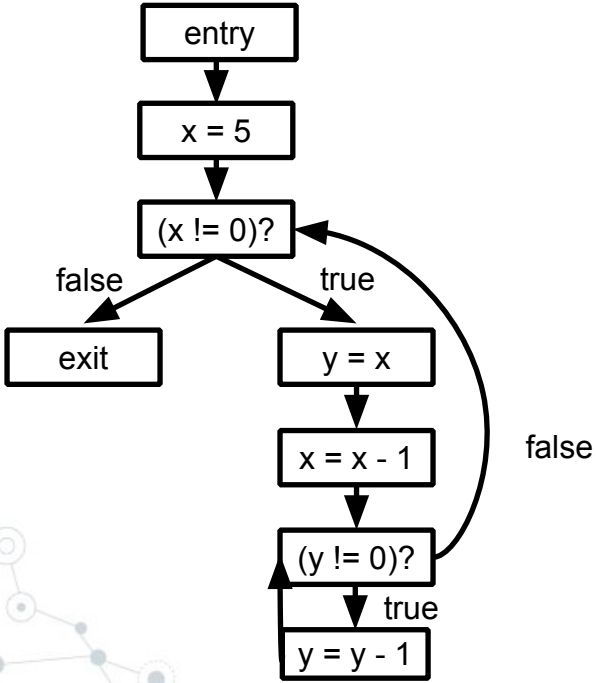
- Control flow graph (CFG):
  - Represents possible control flows with in the function.
  - Graph of basic blocks.
  - **Basic block:** Sequence of instructions always executed in the order.

Control flow graph (CFG)




```
x = 5;
y = 1;
while (x != 1) {
  y = x * y;
  x = x - 1
}
```

Control flow graph (CFG)




```
x = 5;
while (x != 0) {
  y = x;
  x = x - 1;
  while (y != 0) {
    y = y - 1
  }
}
```

Classic Dataflow Analyses -> Primarily used in compiler optimization

Reaching Definitions Analysis

- Find uninitialized variable uses

Very Busy Expressions Analysis

- Reduce code size

Available Expressions Analysis

- Avoid recomputing expressions

Live Variables Analysis

- Allocate registers efficiently

Security related Dataflow Analyses

Interval Analysis

- Check memory safety
  (integer overflows, buffer overruns, …)

Type-State Analysis

- Temporal safety properties
  (APIs of protocols, libraries, …)

Taint Analysis

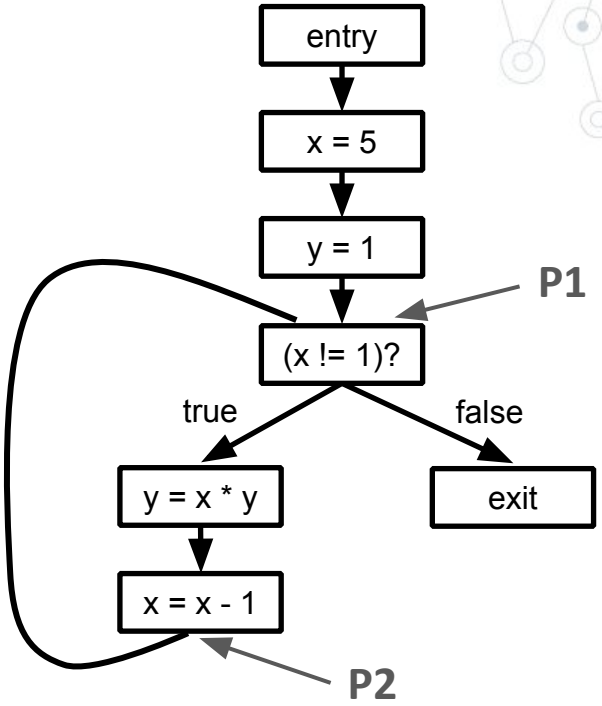
- Check information flow
  (Sensitive data leak, code injection, …)

Concurrency Analysis

- Concurrency safety properties
  (dataraces, deadlocks, …)

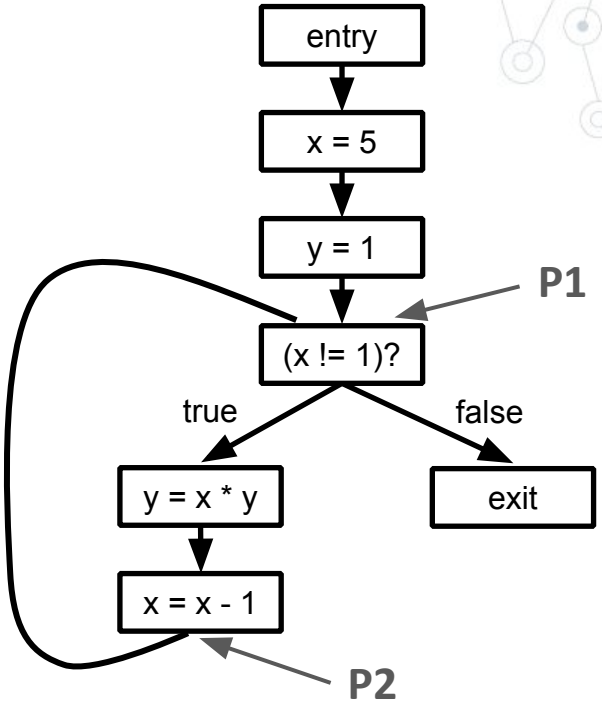
Reaching Definition Analysis

Determine, for each program point, which assignments (definitions) have been made and not overwritten, when execution reaches that point along some path.



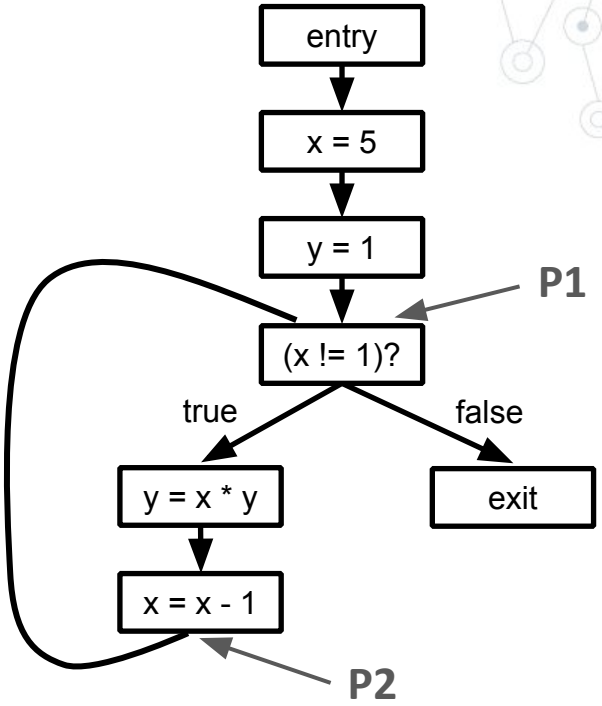
Reaching Definition Analysis

1. The assignment y = 1 reaches P1 ☐
1. The assignment y = 1 reaches P2 ☐
1. The assignment y = x * y reaches P1 ☐



Reaching Definition Analysis

1. The assignment y = 1 reaches P1 ■
1. The assignment y = 1 reaches P2 □
1. The assignment y = x * y reaches P1 ■

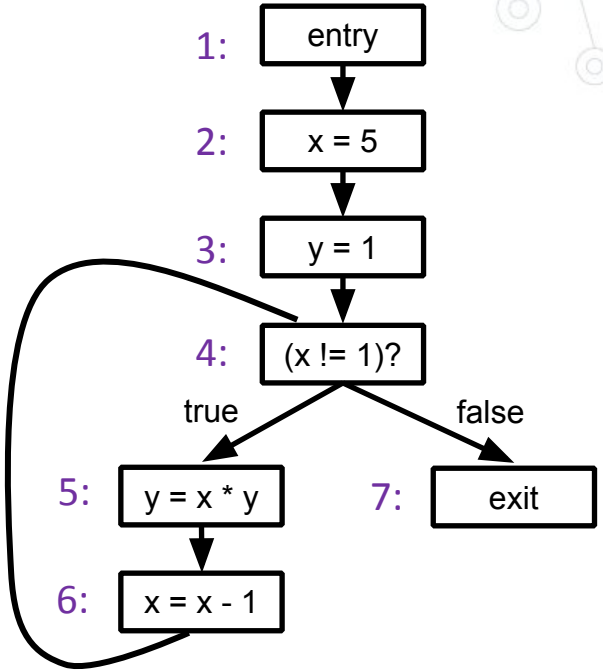


Reaching Definition Analysis

- Result: Set of definitions at each program point
- A definition is a pair of the form:

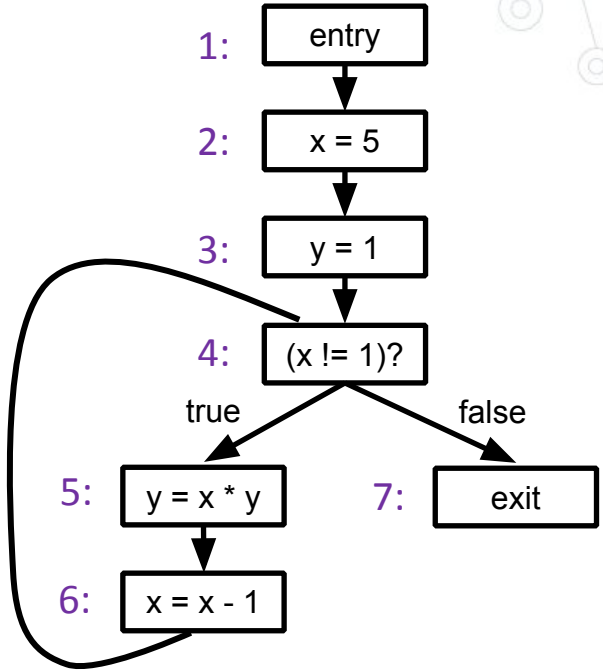
  <defined variable name, defining node label>

- Examples: <x,2> , <y,5>



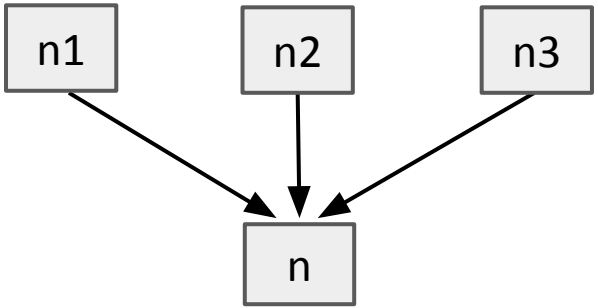
Reaching Definition Analysis

- Give a distinct label n to each node
- IN[n] = set of facts at entry of node n
- OUT[n] = set of facts at exit of node n
- Dataflow analysis computes IN[n] and OUT[n] for each node
- Repeat two operations until IN[n] and OUT[n] stop changing
  - Called “saturated” or “fixed point”



Reaching Definition Analysis: Computing IN

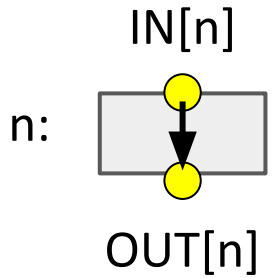
$$IN[n] = \bigcup_{n' \in predecessors(n)} OUT[n']$$



$$IN[n] = OUT[n1] \cup OUT[n2] \cup OUT[n3]$$

Reaching Definition Analysis: Computing OUT

$$OUT[n] = (IN[n] - KILL[n]) \cup GEN[n]$$



n: b ?

$GEN[n] = \varnothing$ $KILL[n] = \varnothing$

n: x = a

$GEN[n] = \{ \langle x, n \rangle \}$

$KILL[n] = \{ \langle x, m \rangle : m \neq n \}$

Overall algorithm: Chaotic Iteration

for (each node n):

IN[n] = OUT[n] = ∅

OUT[entry] = { <v, ?> : v is a program variable }

repeat:

for (each node n):

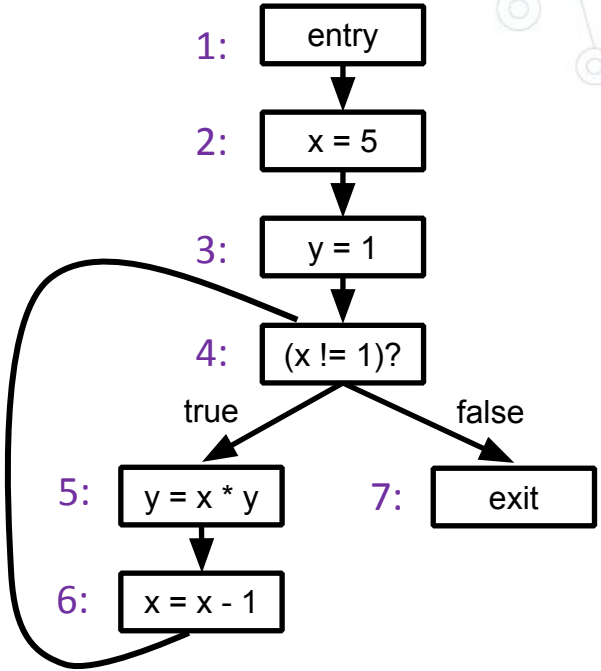
$$IN[n] = \bigcup_{n' \in predecessors(n)} OUT[n']$$

OUT[n] = (IN[n] - KILL[n]) ∪ GEN[n]

until IN[n] and OUT[n] stop changing for all n

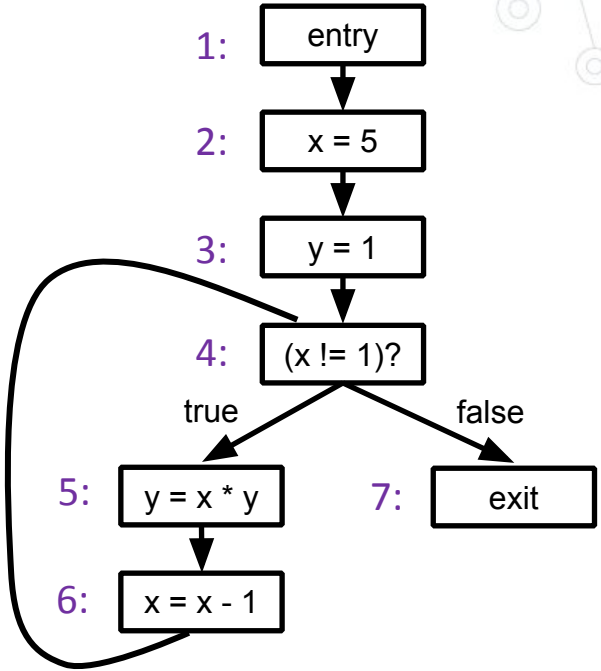
Reaching Definition Analysis: Example

n	IN[n]	OUT[n]
1	--	{<x,?>,<y,?>}
2	∅	∅
3	∅	∅
4	∅	∅
5	∅	∅
6	∅	∅
7	∅	--



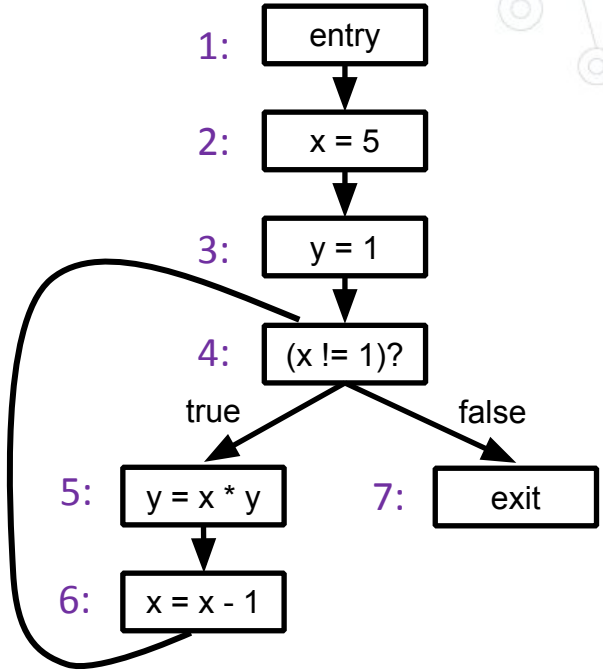
Reaching Definition Analysis: Example

n	IN[n]	OUT[n]
1	--	{<x,?>,<y,?>}
2	{<x,?>,<y,?>}	{<x,2>,<y,?>}
3	{<x,2>,<y,?>}	{<x,2>,<y,3>}
4	∅	∅
5	∅	∅
6	∅	∅
7	∅	--



Reaching Definition Analysis: Example

n	IN[n]	OUT[n]
1	--	{<x,?>,<y,?>}
2	{<x,?>,<y,?>}	{<x,2>,<y,?>}
3	{<x,2>,<y,?>}	{<x,2>,<y,3>}
4	{<x,2>,<y,3>,<y,5>,<x,6>}	{<x,2>,<y,3>,<y,5>,<x,6>}
5	{<x,2>,<y,3>,<y,5>,<x,6>}	{<x,2>,<y,5>,<x,6>}
6	{<x,2>,<y,5>,<x,6>}	{<y,5>,<x,6>}
7	{<x,2>,<y,3>,<y,5>,<x,6>}	--

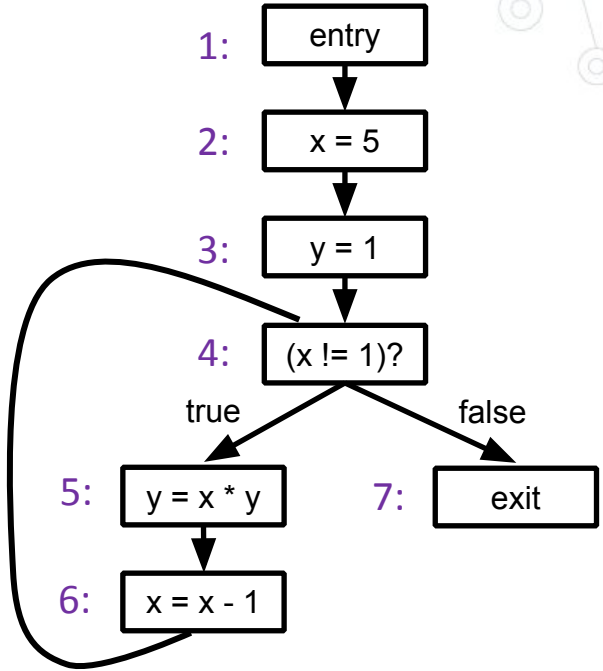


Reaching Definition Analysis: Abstract Domain

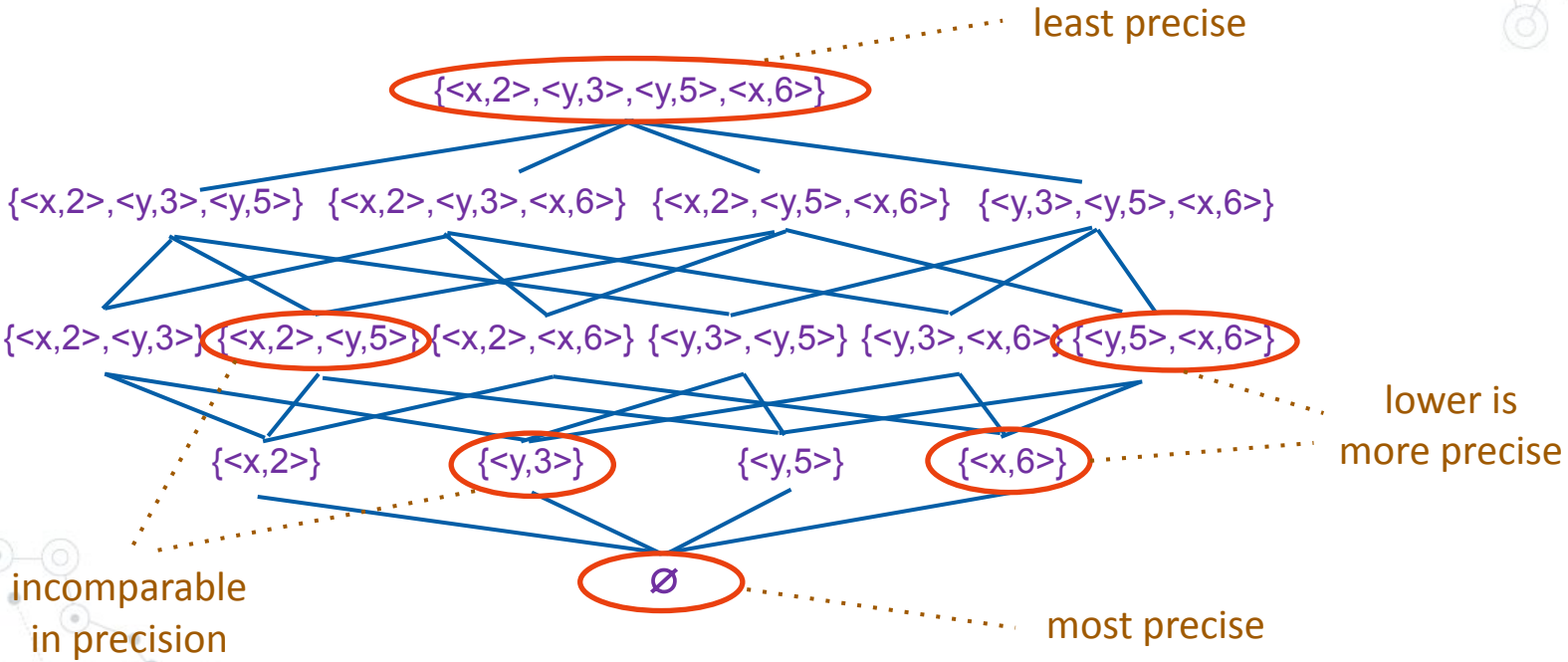
- Any combination of the definitions <x, 2>, <y,3>, <y,5>, <y,6> may reach a particular program point
- So, each combination of definitions is an abstract value
- Abstract domain is:

$\langle 2^{\{ \langle x,2\rangle, \langle y,3\rangle, \langle y,5\rangle, \langle y,6\rangle \}}, \subseteq \rangle$

set inclusion



Reaching Definition Analysis: Abstract Domain



Galois Connection and Termination

Abstract and Concrete domain form Galois connection.

The Chaotic Iteration algorithm always terminates!

- The two operations of reaching definitions analysis are “monotonic”

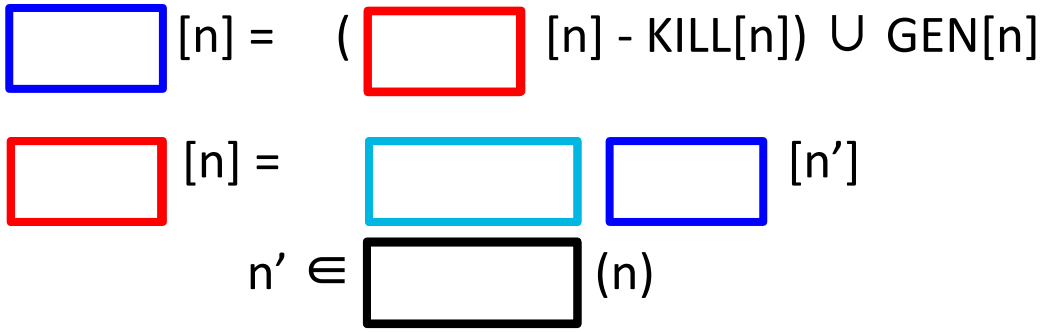
  => IN and OUT sets never shrink, only grow

- Largest they can be is set of all definitions in program, which is finite

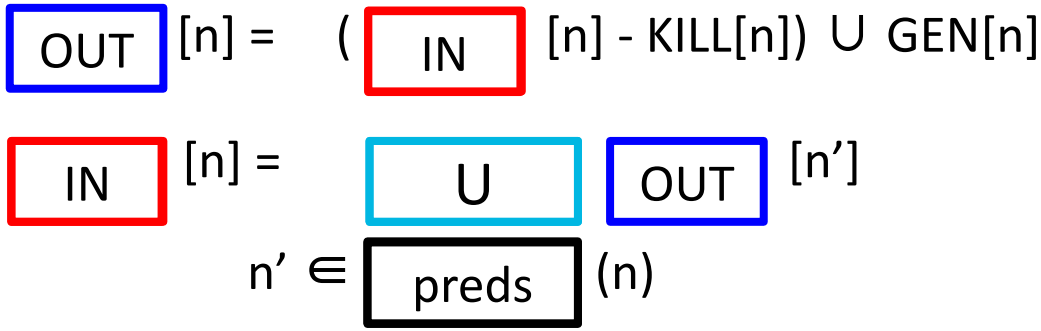
  => IN and OUT cannot grow forever

=> IN and OUT will stop changing after some iteration

Overall Pattern of Dataflow Analysis



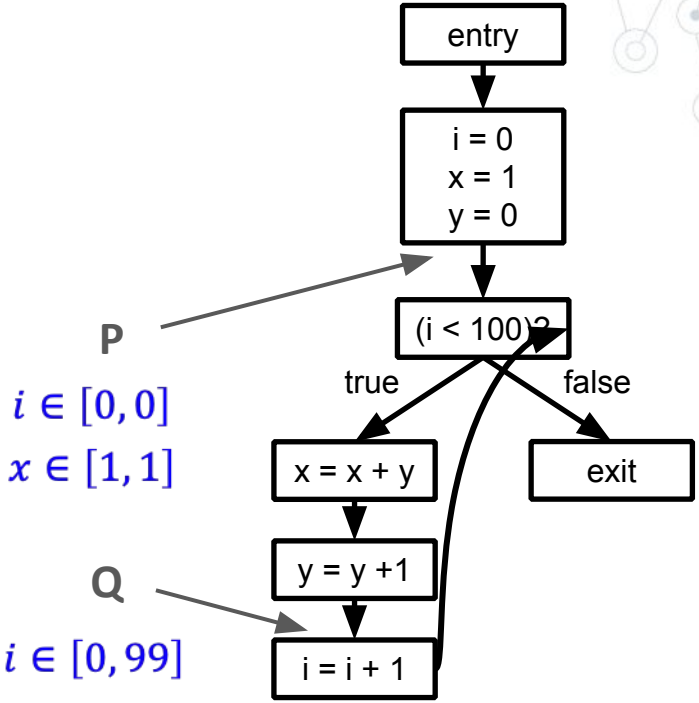
Reaching Definition Analysis



Interval Analysis

Goal: Determine, for each integer variable at each program point, a lower bound and an upper bound on its possible values at that point.

Improving Integer Security for Systems with KINT [OSDI 12]



Uses of Interval Analysis: Integer overflow detection

$in \in [1, +\infty]$

~~$in \in [-\infty, +\infty]$~~

```
void overflow() {
  char *out;
  int in = get_int();          1073741824
  if (in <= 0) { return; }          0
  out = malloc(in*sizeof(char*));
  for (i = 0; i < in; i++)
    out[i] = get_string();
}
```

CVE-2019-3855

In LibSSH, an attacker can exploit to execute code on the client system when a user connects to the server

CVE-2019-8099

In Adobe Acrobat, an attacker can use to steal information

Uses of Interval Analysis: Out of bounds access

$index \in [0, 3]$

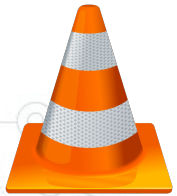
~~$index \in [-\infty, +\infty]$~~

```
int main () {
  char *items[] = {"boat", "car", "truck", "train"};
  int index = get_int();
  if (index < 0 || index > 3) { return; }
  printf("You selected %s\n", items[index]);
}
```

Uses of Interval Analysis: Divide by zero detection

$numRequests \in [-\infty, +\infty]$

```
int averageResponseTime(int totalTime, int numRequests) {
  return totalTime / numRequests;
}
```

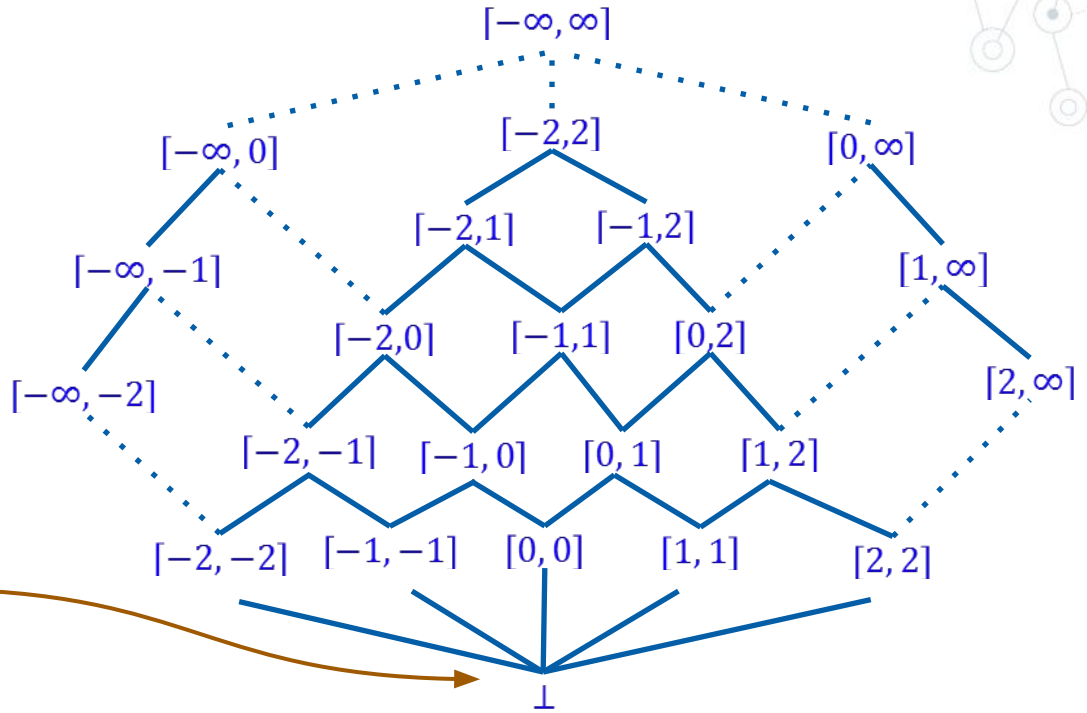


CVE-2019-14498

A divide-by-zero error exists in VLC media player that can be exploited by a crafted audio file

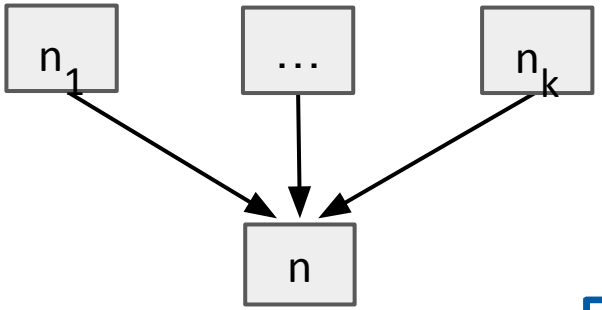
Interval Analysis: Abstract Domain

- Intervals ordered by inclusion
- The lattice has infinite height!



Bottom element interpreted as "not an integer"

Interval Analysis: Computing IN



$$IN[n] = \bigsqcup_{n' \in predecessors(n)} OUT[n']$$

for each variable **x**:

$$IN[n](\mathbf{x}) = [\mathbf{MIN}(l_1, \ldots, l_k), \mathbf{MAX}(h_1, \ldots, h_k)]$$

where:

$$OUT[n_1](\mathbf{x}) = [l_1, h_1]$$

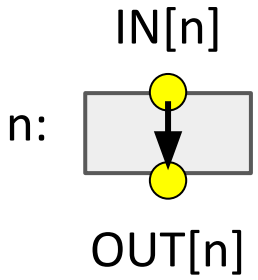
…

$$OUT[n_k](\mathbf{x}) = [l_k, h_k]$$

Interval Analysis: Computing OUT, differs with operations

n: `x = y` OUT[n](x) = IN[n](y)

n: `x = y - z` OUT[n](x) = [y1 - z2, y2 - z1]



where:
- IN[n](y) = [y1, y2]
- IN[n](z) = [z1, z2]

OUT[n](w) = IN[n](w) for each variable w other than x

Interval Analysis: Computing OUT, differs with operations

OUT[n](x) = [y1 + z1 , y2 + z2]

n: x = y + z

where:
- IN[n](y) = [y1, y2]
- IN[n](z) = [z1, z2]

OUT[n](w) = IN[n](w) for each variable w other than x

Interval Analysis: Computing OUT, differs with operations

n: x = y * z

OUT[n](x) = [MIN(x1y1, x1y2, x2y1, x2y2), MAX(x1y1, x1y2, x2y1, x2y2)]

where:

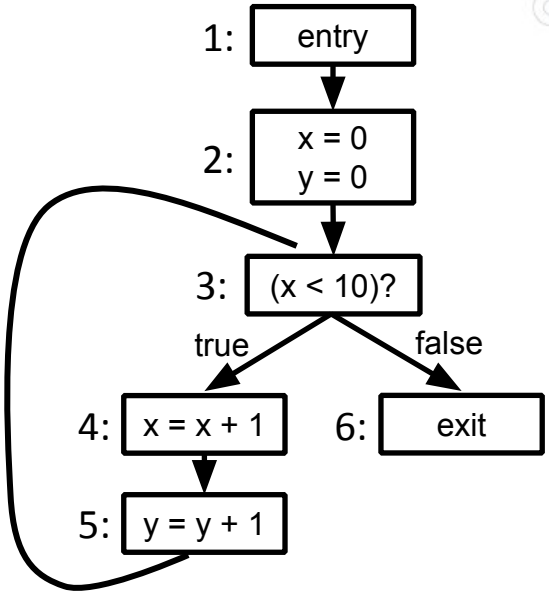
IN[n](y) = [y1, y2]

IN[n](z) = [z1, z2]

OUT[n](w) = IN[n](w) for each variable w other than x

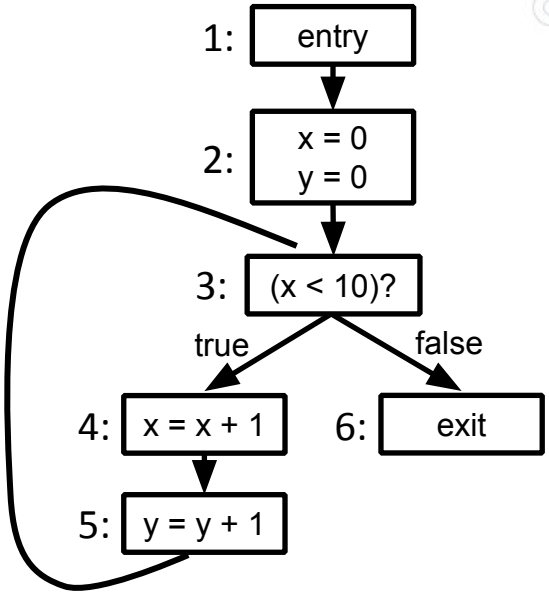
Interval Analysis: Example

n	Iter # 0
1	x ∈[-∞,∞] y ∈[-∞,∞]
2	x ∈ {⊥} y ∈ {⊥}
3	x ∈ {⊥} y ∈ {⊥}
4	x ∈ {⊥} y ∈ {⊥}
5	x ∈ {⊥} y ∈ {⊥}



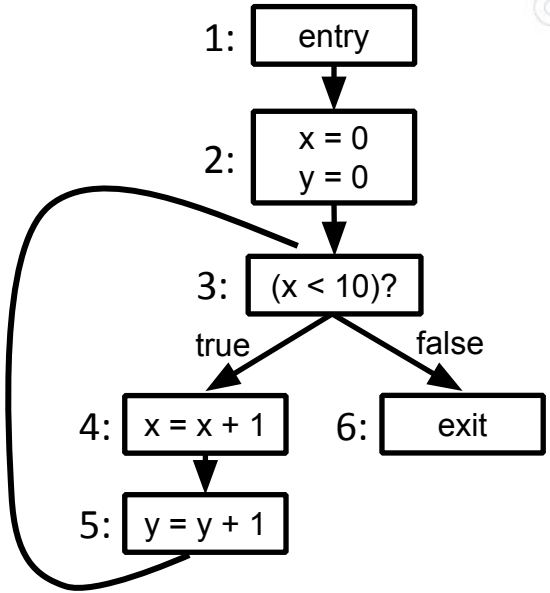
Interval Analysis: Example

n	Iter # 0	Iter # 1
1	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$
2	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [0,0]$ $y \in [0,0]$
3	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [0,0]$ $y \in [0,0]$
4	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [1,1]$ $y \in [0,0]$
5	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [1,1]$ $y \in [1,1]$



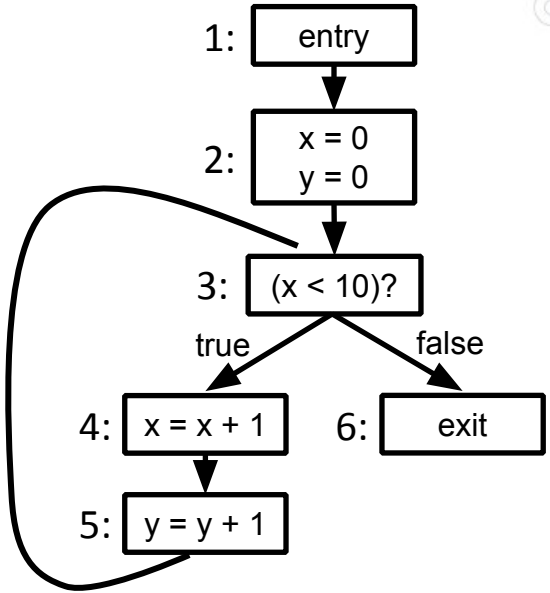
Interval Analysis: Example

n	Iter # 0	Iter # 1	Iter # 2	Iter # 3	Iter # k
1	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$
2	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,0]$ $y \in [0,0]$
3	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,1]$ $y \in [0,1]$	$x \in [0,2]$ $y \in [0,2]$	$x \in [0,k-1]$ $y \in [0,k-1]$
4	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [1,1]$ $y \in [0,0]$	$x \in [1,2]$ $y \in [0,1]$	$x \in [1,3]$ $y \in [0,2]$	$x \in [1,k]$ $y \in [0,k-1]$
5	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [1,1]$ $y \in [1,1]$	$x \in [1,2]$ $y \in [1,2]$	$x \in [1,3]$ $y \in [1,3]$	$x \in [1,k]$ $y \in [1,k]$



Interval Analysis: Example

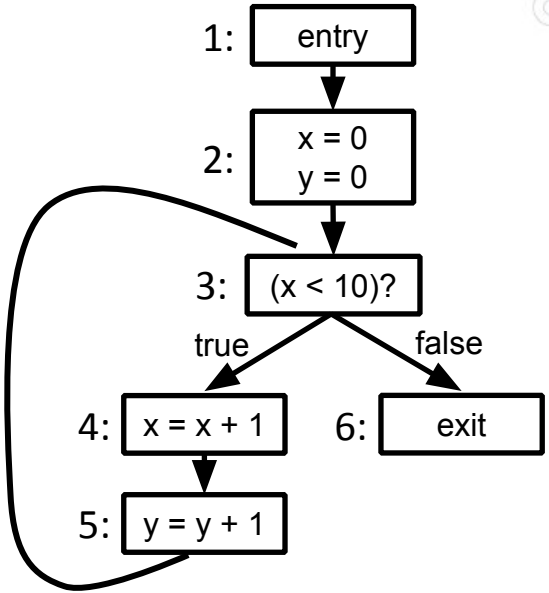
n	Iter # 0	Iter # 1	Iter # 2	Iter # 3	Iter # k
1	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$	$x \in [-\infty,\infty]$ $y \in [-\infty,\infty]$
2	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,0]$ $y \in [0,0]$
3	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [0,0]$ $y \in [0,0]$	$x \in [0,1]$ $y \in [0,1]$	$x \in [0,2]$ $y \in [0,2]$	$x \in [0,k-1]$ $y \in [0,k-1]$
4	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [1,1]$ $y \in [0,0]$	$x \in [1,2]$ $y \in [0,1]$	$x \in [1,3]$ $y \in [0,2]$	$x \in [1,k]$ $y \in [0,k-1]$
5	$x \in \{\bot\}$ $y \in \{\bot\}$	$x \in [1,1]$ $y \in [1,1]$	$x \in [1,2]$ $y \in [1,2]$	$x \in [1,3]$ $y \in [1,3]$	$x \in [1,k]$ $y \in [1,k]$



Interval Analysis: Example

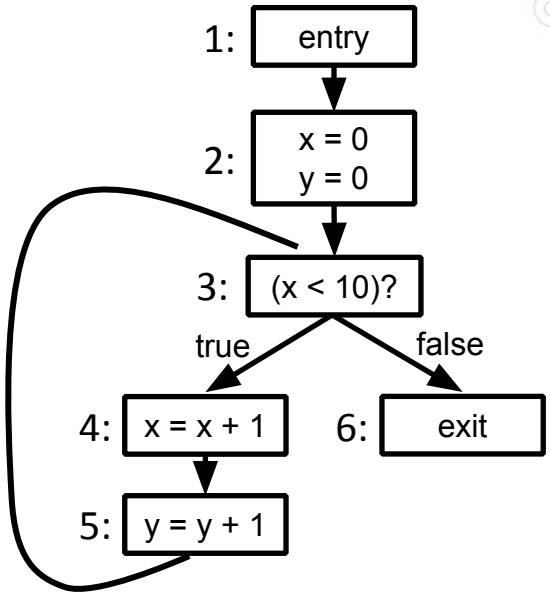
n	Iter # 0	Iter # 1	Iter # 2	Iter # 3	Iter # k
1	x ∈[-∞,∞] y ∈[-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]
2	x ∈ {⊥} y ∈ {⊥}	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,0] y ∈ [0,0]
3	x ∈ {⊥} y ∈ {⊥}	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,1] y ∈ [0,1]	x ∈ [0,2] y ∈ [0,2]	x ∈ [0,**9**] y ∈ [0,9]
4	x ∈ {⊥} y ∈ {⊥}	x ∈ [1,1] y ∈ [0,0]	x ∈ [1,2] y ∈ [0,1]	x ∈ [1,3] y ∈ [0,2]	x ∈ [1,**10**] y ∈ [0,9]
5	x ∈ {⊥} y ∈ {⊥}	x ∈ [1,1] y ∈ [1,1]	x ∈ [1,2] y ∈ [1,2]	x ∈ [1,3] y ∈ [1,3]	x ∈ [1,**10**] y ∈ [1,10]

more precise analysis



Interval Analysis: Example

n	Iter # 0	Iter # 1	Iter # 2	Iter # 3	Iter # k	Iter # ∞
1	x ∈[-∞,∞] y ∈[-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]	x ∈ [-∞,∞] y ∈ [-∞,∞]
2	x ∈ {⊥} y ∈ {⊥}	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,0] y ∈ [0,0]
3	x ∈ {⊥} y ∈ {⊥}	x ∈ [0,0] y ∈ [0,0]	x ∈ [0,1] y ∈ [0,1]	x ∈ [0,2] y ∈ [0,2]	x ∈ [0,k-1] y ∈ [0,k-1]	x ∈ [0,∞] y ∈ [0,∞]
4	x ∈ {⊥} y ∈ {⊥}	x ∈ [1,1] y ∈ [0,0]	x ∈ [1,2] y ∈ [0,1]	x ∈ [1,3] y ∈ [0,2]	x ∈ [1,k-1] y ∈ [0,k-1]	x ∈ [1,∞] y ∈ [0,∞]
5	x ∈ {⊥} y ∈ {⊥}	x ∈ [1,1] y ∈ [1,1]	x ∈ [1,2] y ∈ [1,2]	x ∈ [1,3] y ∈ [1,3]	x ∈ [1,k-1] y ∈ [1,k-1]	x ∈ [1,∞] y ∈ [1,∞]



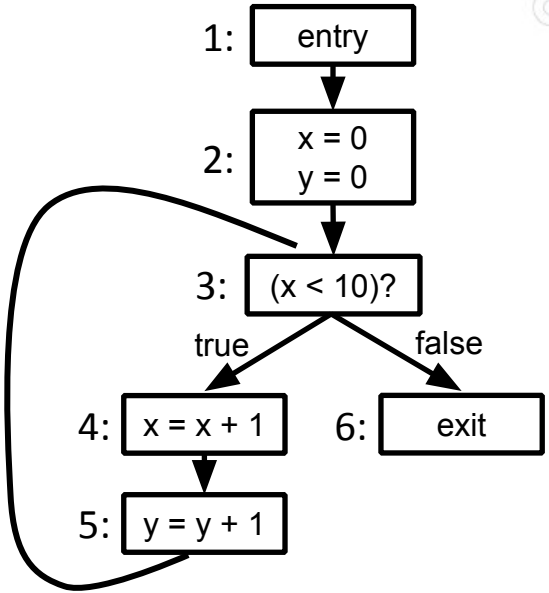
Interval Analysis: Example

- In infinite-height lattice, the fixed-point computation does not terminate!
- Solution: Widening

Infinite ascending chain:

$y \rightarrow \bot, [1,1], [1,2], [1,3], \ldots, [1,k-1], \ldots, [1,\infty]$

Finite ascending chain: $y \rightarrow \bot, [1,1], [1,2], [1,\infty]$



What can go wrong in a securely designed IoT system?

We designed an IoT system securely with a strong threat model.

What can go wrong?